Running the `dihiggs` notebooks on the RTX 5090

A short manual: long runs that survive a broken connection,
with a `tmux` cheat sheet

October 31, 2026 (first version October 30)

Abstract

The BNN and ensemble notebooks run for an hour or more on the 5090, reached from a laptop by `ssh`. This note explains which parts of the set-up live where, what survives a closed browser tab or a broken connection, and how to start, watch, interrupt, resume and collect a run. Names in angle brackets, such as `<user>@<host>`, stand for your own values. The conda environment on the 5090 is `ml`; the repository is `~/Projects/workbench/dihiggs` there and `~/Projects/Tutorials/GitHub/dihiggs` on the Mac.

The one rule

Everything that must keep running is started, directly or indirectly, inside `tmux` on the 5090. Then the laptop, the browser and the `ssh` connection can all go away without stopping it.

1 What runs where

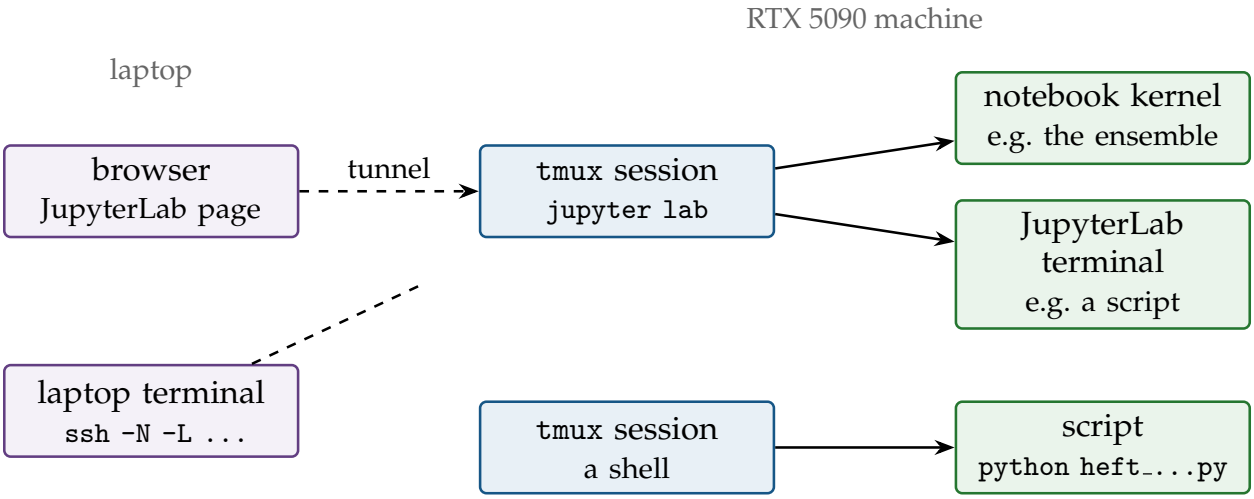


Figure 1: Purple: on the laptop; they may disappear at any time. Blue and green: on the 5090. Kernels and JupyterLab terminals belong to the Jupyter server; scripts started in their own `tmux` session depend on nothing else.

event	the run in Jupyter	a script in `tmux`
close the browser tab or the browser	continues	continues
break the tunnel, close the laptop, lose the network	continues	continues
JupyterLab: close a notebook or terminal *tab*	continues	continues
JupyterLab: **Shut Down** kernel/terminal, **File** → **Shut Down**	**stops**	continues
`Ctrl-c` in the `tmux` pane running `jupyter lab`	**stops**	continues
`Ctrl-c` in the `tmux` pane running the script	continues	**stops**
the 5090 reboots	**stops**	**stops**

While no browser is connected, the kernel's printed output is not recorded in the notebook. The computation and every file it writes (run folders, traces, figures) are unaffected.

2 Who owns a process: the rule behind everything

Every process on the 5090 has a parent, and what happens to it when you disconnect depends on that parent. When an `ssh` session ends, normally or because the connection drops, the system sends its shell a *hang-up* signal (SIGHUP), and the shell passes it on to the jobs it started, which then stop. A process survives if its parent does not hang up, or if it has been told to ignore the signal.

started from	belongs to	survives logout?
a notebook in JupyterLab	the Jupyter server (in `tmux`)	yes
a JupyterLab terminal	the Jupyter server (in `tmux`)	yes
a login shell, `command &`	your login session	**no**
a login shell, `nohup command &`	nobody who will hang up	yes
a shell inside `tmux`	the `tmux` server	yes

The browser page on the laptop is only a window onto JupyterLab, and the tunnel only the line to it: closing either touches nothing on the 5090.

`nohup`. `nohup bash run.sh > /dev/null 2>&1 &` starts the script in the background, immune to the hang-up signal. `> /dev/null 2>&1` discards what the script prints to the terminal; if the script itself writes a log (with `tee`), nothing is lost. Use it for a one-off job from a login shell. The disadvantage: you cannot come back to the job's terminal later; you can only read its log.

3 The `tmux` programming model

- **The server** starts automatically with the first `tmux` command and runs until its last session ends (or the machine reboots). It is the parent of every shell in every window, which is why those shells, and the programs they run, survive your logging out.
- **A session** is a named group of windows, e.g. `work`. You attach to a session from any terminal (an `ssh` login today, another one tomorrow) and detach from it at will; it does not care which terminal, if any, is looking at it.
- **A window** is one full-screen shell, numbered 0, 1, 2, . . . and shown in the status bar at the bottom. Use one window per job: e.g. window 0 for `jupyter lab`, window 1 for a fine-tune, window 2 for watching logs.

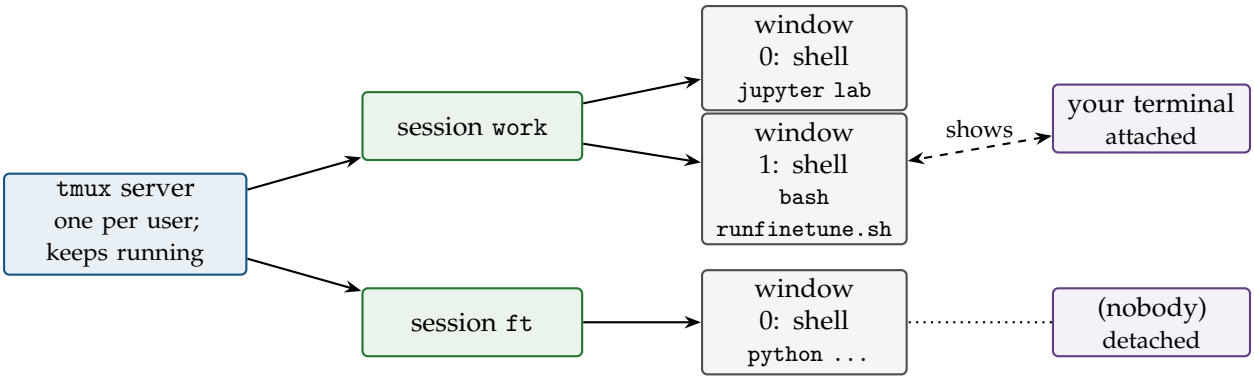


Figure 2: `tmux`'s structure. The server owns sessions, sessions own windows, and each window runs a shell, in which your programs run. A terminal *attached* to a session shows one of its windows; *detaching* disconnects the terminal and leaves everything running.

- **A pane** is a window split into parts, each with its own shell. Optional; handy for watching `nvidia-smi` beside a log.
- **The prefix key.** `tmux` commands typed *inside* a session start with the prefix `Ctrl-b`: press and release `Ctrl-b`, then the command key. For example `Ctrl-b` then `d` detaches.
- **Ending things.** `exit` in a window's shell closes that window (and the session, when its last window closes). `Ctrl-c` stops the program running in the current window, not `tmux`. Neither should ever be used in the window running `jupyter lab` unless you mean to stop JupyterLab.

Adding a job to a session that is already running

Attach (`tmux attach -t work`), open a new window (`Ctrl-b c`), start the job there with no `&` or `nohup` (the window's shell belongs to `tmux`), and detach (`Ctrl-b d`). The new job and everything already running continue.

4 `tmux` cheat sheet

Shell commands (typed at a prompt, outside or inside `tmux`).

command	what it does
`tmux new -s work`	start a new session called `work` and attach to it
`tmux ls`	list the sessions
`tmux attach -t work`	attach to session `work`
`tmux attach`	attach to the most recent session
`tmux new-window -t work`	add a window to `work` without attaching
`tmux kill-session -t work`	end a session and everything running in it

Keys inside a session (press `Ctrl-b`, release, then the key).

Ctrl-b, then	what it does
d	detach: leave everything running and return to the plain shell
c	create a new window
n / p	next / previous window
0 … 9	go to window 0 … 9
w	choose a window (or session) from a list
,	rename the current window (e.g. jupyter, finetune)
% / "	split the window into two panes, side by side / one above the other
o	move to the next pane
x	close the current pane (asks for confirmation)
[	scroll mode: look back through the output with the arrow keys or PageUp/-PageDown; q leaves it
?	list all key bindings

A typical day.

```
ssh <user>@<host>
tmux ls
tmux attach -t work
```

Then Ctrl-b w to see the windows, Ctrl-b c for a new one, run the job, Ctrl-b d to leave. If tmux ls says there is no server, nothing is running in tmux: start with tmux new -s work.

Three slips to avoid

1. Typing a new job into the window running jupyter lab: that window is busy, and Ctrl-c there stops JupyterLab. Open a new window instead.
2. Starting tmux inside tmux (running tmux new from a window of an attached session): tmux warns about nesting; use Ctrl-b c for another window instead.
3. Closing the laptop's terminal *without* detaching is harmless: the session simply becomes detached. But exit typed in a window closes that window, and whatever it was running stops.

5 Updating the code first

On the Mac, commit and push; on the 5090, pull:

```
cd ~/Projects/Tutorials/GitHub/dihiggs
git push
```

```
cd ~/Projects/workbench/dihiggs
git pull
```

Then restart any Jupyter kernel that imports dihiggs, so that it loads the new modules. If git pull refuses because a notebook was edited on the 5090 (for example its switches), discard that edit and pull again, then re-enter the switches:

```
git checkout code/heft_train_bnn.ipynb
git pull
```

A quick check that the 5090 has the new `hmc.py`: a BNN run prints, at the start, a line beginning `warm-up:  step size only for iterations 1-75`.

6 Recipe A: a script in its own `tmux` session (most robust)

1. Set the switches in the notebook (in JupyterLab, or any editor) and save it.
2. On the 5090, start a named `tmux` session (or, if one is already running, attach to it and open a new window with `Ctrl-b c`; section 3) and go to the code folder:

```
tmux new -s bnn
conda activate ml
cd ~/Projects/workbench/dihiggs/code
```

3. Convert the notebook to a script, and run it:

```
jupyter nbconvert --to python heft_train_bnn.ipynb
MPLBACKEND=Agg python heft_train_bnn.py 2>&1 | tee bnn_run.log
```

   `--to python` always writes a `.py` file (`--to script` takes the extension from the notebook's metadata, and for a notebook never saved in Jupyter may write a `.txt` file instead). `MPLBACKEND=Agg` makes matplotlib write figures to files instead of opening windows; `tee` shows the output and also keeps it in `bnn_run.log`.

4. Detach: press `Ctrl-b`, then `d`. You may now log out.
5. Later, reattach with

```
tmux attach -t bnn
```

The same works for `heft_train_ensemble.ipynb` (choose another session name, e.g. `ens`, and another log file). Re-run the `nbconvert` line whenever the notebook changes, or the script will run the old switches.

7 Recipe B: JupyterLab through an `ssh` tunnel

1. On the 5090, start the Jupyter server inside `tmux`, then detach (`Ctrl-b`, `d`):

```
tmux new -s jupyter
conda activate ml
cd ~/Projects/workbench/dihiggs
jupyter lab --no-browser --port <port>
```

   Note the URL with its token that it prints.

2. On the laptop, open the tunnel in a terminal of its own, and leave it open:

```
ssh -N -L <port>:localhost:<port> <user>@<host>
```

   Then open `http://localhost:<port>` (with the token) in the browser.

3. Run the notebook as usual, or open a JupyterLab terminal and run Recipe A's script commands there (step 3).

4. To leave: close the browser tab, then press `Ctrl-c` in the tunnel's terminal (or close it). Never use the **Shut Down** entries of JupyterLab for a run you want to keep.
5. To come back: open the tunnel again and reload the page.

A script started in a JupyterLab terminal

It runs on the 5090 and survives a closed tab and a broken tunnel, but it belongs to the Jupyter server: if the server stops, so does the script. For long runs, Recipe A is safer.

8 Watching a run

From any `ssh` session on the 5090, in `~/Projects/workbench/dihiggs/code`: the script's output so far, the BNN trace (one line per iteration), the ensemble summary (one line per member), the GPU, and the running `tmux` sessions:

```
cd ~/Projects/workbench/dihiggs/code
tail -n 30 bnn_run.log
tail -f runs/BNN_poisson/dihiggs_hmc_trace.csv
cat runs/ENS_bootstrap/ensemble.csv
nvidia-smi
tmux ls
```

`tail -f` keeps printing new lines; `Ctrl-c` stops the watching, not the run.

9 Interrupting and resuming

- **BNN.** Stop a run with `Ctrl-c` in its `tmux` pane, or **Kernel** → **Interrupt** in JupyterLab. The sampler then saves a checkpoint at the last complete iteration (it also saves one every few minutes, in case of a crash). To continue, set `RESUME = True`, keep the other switches, save, and run again (for Recipe A, re-run the `nbconvert` line first). The result is identical to an uninterrupted run. A larger `N_SAMPLES` extends a finished run.
- **Ensemble.** Members already finished are skipped, so just run the notebook again; at most the member in progress is lost.
- **Before resuming,** make sure the old run has really stopped (`tmux ls`, `nvidia-smi`): two processes writing to the same run folder would corrupt it.

10 Several jobs on one GPU

`HEFTNet` is small: each job needs about 0.5–1 GB of the 5090's 32 GB, and much of its time is Python overhead, so two or three jobs can share the GPU, each running somewhat slower. Check `nvidia-smi`: if the utilisation is already near 100%, another job mainly slows the others down. Each job also keeps about one CPU core busy. Two copies of the ensemble notebook can train different members at once, with disjoint `MEMBERS` ranges such as `range(0, 10)` and `range(10, 20)`.

11 Bringing the results back to the Mac

On the 5090, pack the run folder; on the Mac, copy it into `code/runs/`:

```
cd ~/Projects/workbench/dihiggs/code/runs
tar czf BNN_poisson.tar.gz BNN_poisson
```

```
cd ~/Projects/Tutorials/GitHub/dihiggs/code/runs
scp <user>@<host>:Projects/workbench/dihiggs/code/runs/BNN_poisson.tar.gz .
tar xzf BNN_poisson.tar.gz
```

Run folders are overwritten

A new BNN run with the same likelihood writes into the same folder, `runs/BNN_<likelihood>`, and replaces the previous run. Rename the old folder first (e.g. `BNN_poisson` → `BNN_poisson3`), on the 5090 before starting, and on the Mac before unpacking.